inacap
LABORATORIO CENTRAL · CLASE 3

Estudiante: Luckas Araya | Carrera: Analista Programador | Sede: La Serena

Mejora del diagrama UML de EcoTech Solutions

Campo	Información
Asignatura	Programación Orientada a Objetos · Unidad 1
Estudiante	Luckas André Araya Mancilla
Actividad	Clase 3, diapositiva 13: Laboratorio central
Entregable	UML individual actualizado + dos decisiones justificadas
Estado	Documentación del avance; adjuntar UML revisado antes de publicar

1. Propósito y punto de partida

La diapositiva 13 pide retomar el modelo de la Clase 2, seleccionar una necesidad del enunciado aún no resuelta claramente, identificar información y acciones, examinar asociación o generalización, revisar una multiplicidad y un método, y proponer una mejora con un caso que permita comprobarla. No se copian clases del docente.

Punto de partida: el modelo previo de EcoTech representaba Departamento, Empleado, Proyecto, RegistroTiempo e Informe, sus asociaciones y operaciones. La guía de la evaluación también exige autenticación, autorización y protección de la información. Esta necesidad de seguridad no estaba representada con la misma claridad en el UML anterior.

2. Desarrollo de los cinco pasos del laboratorio

Paso 01 · Necesidad no resuelta claramente

Necesidad seleccionada: controlar quién ingresa al sistema y a qué módulos puede acceder. El registro de empleados y la generación de informes no deben equivaler, por sí solos, a tener permisos. La mejora responde al requisito de autenticación y autorización de la guía EcoTech.

Paso 02 · Información y acción necesarias

Clase	Información / atributos	Acciones / métodos
CuentaUsuario	nombreUsuario: String; hashContrasena: String; sal: String; rol: String	verificarContrasena(contrasena: String): Boolean; obtenerNombreUsuario(): String; obtenerRol(): String
ControlAcceso	usuarioAutenticado: String	autenticarUsuario(nombreUsuario: String, contrasena: String): Boolean; autorizarAcceso(modulo: String): Boolean; cerrarSesion(): Void

Justificación: CuentaUsuario concentra la información de identidad y credenciales; ControlAcceso concentra la comprobación de identidad, la decisión de autorización y el estado de sesión. El hash y la sal no se publican mediante accesores. No se afirma que exista una implementación funcional ni se especifica un algoritmo de hashing que no esté acreditado.

Paso 03 · Asociación o generalización: decisión y fundamento

Se incorporan dependencias de uso: ControlAcceso → CuentaUsuario («consulta credenciales») e Informe → ControlAcceso («solicita autorización»). En UML se representan con línea discontinua y flecha abierta; no se les asignan multiplicidades. No se añade generalización porque una cuenta no «es un» control de acceso y un control de acceso no «es un» empleado. No hay relación de herencia válida entre estas responsabilidades.

La posible asociación CuentaUsuario–Empleado y su cardinalidad permanecen sin definir: no se ha decidido si todos los empleados o solo algunos dispondrán de cuenta. No se dibuja una relación inventada para cubrir ese vacío.

Paso 04 · Revisión de multiplicidad y responsabilidad

Relación revisada: «Es administrado por», entre Departamento y Empleado. La multiplicidad 1 junto a Empleado significa que cada Departamento es administrado por exactamente un Empleado registrado. La multiplicidad 0..1 junto a Departamento significa que un Empleado administra cero o un Departamento como máximo. La pertenencia a un departamento no equivale a su administración.

Se conserva gerente: String en Departamento y la asociación con Empleado, por decisión del estudiante. El nombre almacenado debe mantenerse coherente con el empleado designado; esa sincronización es una consideración de implementación, no una funcionalidad demostrada.

Responsabilidad revisada: autorizarAcceso(modulo: String): Boolean pertenece a ControlAcceso. Informe solicita la autorización, pero no decide por sí mismo si una cuenta tiene permisos.

Paso 05 · Mejora y caso de comprobación

Mejora propuesta: conservar las cinco clases originales y añadir CuentaUsuario y ControlAcceso, para un total de siete clases, sin rediseñar las relaciones del dominio. El UML actualizado muestra la separación entre identidad, autorización y generación de informes.

Escenario conceptual	Resultado esperado
Cuenta administradora autenticada solicita generar un informe	ControlAcceso comprueba autorización; acceso permitido si el rol posee permiso.
Cuenta sin sesión intenta generar un informe	Acceso denegado.
Cuenta autenticada sin permiso solicita módulo restringido	Acceso denegado.
Empleado registrado no designado gerente	Puede existir sin administrar departamentos.
Departamento creado con gerente registrado	Tiene exactamente un empleado gerente; ese empleado no administra otro departamento.

Los escenarios son pruebas conceptuales del diseño, no resultados de ejecución en Python. Para validar su funcionamiento real se requerirá implementación y pruebas posteriores.

3. Dos decisiones justificadas del estudiante

DECISIÓN 1 — Separación de seguridad. Elegí representar CuentaUsuario y ControlAcceso como dos clases porque los datos de las credenciales y la lógica de autorización cumplen funciones distintas. Así puedo verificar identidad y permisos sin guardar contraseñas en Empleado ni mezclar las reglas de acceso con Informe.

DECISIÓN 2 — Multiplicidad de gerencia. Mantengo gerente: String y la relación «Es administrado por» porque quiero representar tanto el nombre del responsable como su condición de empleado registrado. Cada departamento tiene un gerente y cada empleado puede administrar como máximo un departamento; pertenecer a un departamento es otra relación distinta.

Estas formulaciones están basadas en decisiones comunicadas por el estudiante. Se recomienda revisarlas y adaptarlas a su redacción personal antes de entregar.

4. Revisión crítica con inteligencia artificial

La diapositiva 14 propone usar IA como auditora, no como sustituto del autor. Preguntas de comprobación para una revisión real: (1) ¿Las multiplicidades de «Es administrado por» expresan correctamente un gerente por departamento y máximo un departamento por empleado? (2) ¿La autorización está asignada a ControlAcceso y no confundida con la generación del informe? Posible inconsistencia a vigilar: mantener gerente: String junto a una relación con Empleado puede producir datos desactualizados.

No se presenta este apartado como transcripción de un prompt ya ejecutado. Para conservar evidencia verificable, adjuntar el prompt auténtico y la respuesta obtenida si se realiza la ronda de auditoría de la diapositiva 14.

5. Evidencias del avance y publicación

FIGURA 1 — UML individual actualizado, vista principal (insertar captura exportada de diagrams.net).

FIGURA 2 — Dependencias de seguridad e informes (opcional; insertar captura).

Archivo editable del UML (.drawio): [PEGAR ENLACE DIRECTO DESPUÉS DEL PUSH]

Repositorio GitHub: https://github.com/AtezDev/Ecotech_Poos_Flex

Commit de esta actividad: [PEGAR HASH REAL DESPUÉS DEL PUSH]

Versión de referencia: EcoTech_UML_Luckas Araya_v4c_gerencia_aprobada.drawio. Confirmar visualmente el archivo definitivo antes de publicarlo; esta documentación no sustituye la comprobación de flechas, firmas y multiplicidades en diagrams.net.

6. Conclusión y alcance

La mejora de la Clase 3 hace explícito el requisito de autenticación y autorización mediante dos responsabilidades diferenciadas, revisa una multiplicidad del modelo existente y conserva la arquitectura original. Se justifica no usar herencia sin una relación «es un». El resultado es una propuesta UML conceptual: no acredita que los permisos, los informes o el almacenamiento de contraseñas ya estén implementados.

Fuentes: Guía de Evaluación Sumativa Unidad 1, caso EcoTech Solutions; presentación «Clase 3: Polimorfismo y Relaciones UML», diapositivas 13 y 14; modelo UML v4c del estudiante y decisiones expresadas durante la revisión.